

El equipo investiga como se debe aplicar conceptos de seguridad en las distintas etapas del desarrollo de software en un ciclo de vida, detalla mediante un cuadro cuales son las buenas prácticas que se deben tener en cuenta.

El ciclo de vida de desarrollo del software seguro (SSDLC)

La implementación de procesos de seguridad eficaces requiere que los equipos apliquen el enfoque "shift left", es decir, que apliquen las medidas de protección desde las primeras etapas del SDLC y en cada una de ellas.

Planificación	• Evaluar los riesgos y el panorama de las amenazas a la seguridad • Evaluar el posible impacto de los incidentes de seguridad, como el riesgo para la reputación de la empresa
Análisis de los requisitos	• Incluir los requisitos de seguridad • Comprender e incorporar los requisitos de cumplimiento normativo
Diseño	• Elaborar modelos de amenazas • Incluir las consideraciones de seguridad como parte integral del plan de arquitectura • Evaluar el impacto que tiene la seguridad en las decisiones de la etapa de diseño, como la plataforma y la interfaz de usuario
Desarrollo	• Capacitar a los desarrolladores sobre las prácticas de codificación seguras • Incorporar herramientas de prueba de la seguridad en el proceso de desarrollo • Evaluar las dependencias del software y reducir los riesgos de seguridad
Documentación	• Documentar los procesos y los controles de seguridad • Reunir la información para prepararse para las auditorías, los controles de

	cumplimiento normativo y las revisiones de seguridad
Pruebas	• Implementar procesos de revisión del código • Realizar pruebas estáticas o dinámicas de la seguridad de las aplicaciones
Implementación	• Evaluar la seguridad del entorno de implementación • Revisar las configuraciones de seguridad
Mantenimiento	• Supervisar el sistema para detectar amenazas • Prepararse para eliminar los puntos vulnerables y solucionar las intrusiones

Buenas prácticas

Etapas del SSDLC	Buenas prácticas de seguridad
Planificación	• Evaluar los riesgos y el panorama de amenazas de seguridad. • Evaluar el posible impacto de los incidentes de seguridad, incluyendo riesgos económicos y reputacionales.
Análisis de requisitos	• Incluir requisitos de seguridad desde el inicio del proyecto. • Identificar los datos sensibles que manejará el sistema. • Comprender e incorporar los requisitos legales y de cumplimiento normativo.
Diseño	• Elaborar modelos de amenazas (<i>Threat Modeling</i>). • Incorporar la seguridad como parte integral de la arquitectura. • Evaluar el impacto de la seguridad en las decisiones de diseño, como la plataforma, infraestructura y la interfaz de usuario.

	• Aplicar principios como mínimo privilegio, defensa en profundidad y separación de responsabilidades.
Desarrollo	• Capacitar a los desarrolladores en prácticas de codificación segura. • Incorporar herramientas de análisis y pruebas de seguridad al proceso de desarrollo. • Validar y sanitizar las entradas proporcionadas por los usuarios. • Evaluar las dependencias y librerías utilizadas y mantenerlas actualizadas. • Evitar almacenar contraseñas y secretos de forma insegura.
Documentación	• Documentar los procesos y controles de seguridad implementados. • Mantener documentación sobre requisitos, riesgos y decisiones de seguridad. • Reunir información necesaria para auditorías, controles de cumplimiento y revisiones de seguridad.
Pruebas	• Implementar revisiones de código (<i>Code Review</i>). • Realizar pruebas de seguridad estáticas (SAST) y dinámicas (DAST). • Realizar análisis de vulnerabilidades. • Comprobar mecanismos de autenticación y autorización. • Corregir las vulnerabilidades detectadas antes de pasar a producción.
Implementación	• Evaluar la seguridad del entorno de implementación. • Revisar las configuraciones de seguridad. • Proteger las credenciales, claves y otros secretos utilizados por la aplicación. • Utilizar comunicaciones seguras mediante HTTPS/TLS. • Aplicar permisos y accesos siguiendo el principio de mínimo privilegio.
Mantenimiento	• Supervisar el sistema para detectar amenazas y comportamientos anómalos. • Aplicar actualizaciones y parches de seguridad. • Gestionar y corregir las vulnerabilidades descubiertas después de la implementación. • Contar con mecanismos de respuesta ante incidentes.

	• Realizar copias de seguridad y establecer mecanismos de recuperación.
--	---

Dar ejemplos de aplicaciones inseguras que no utilizan seguridad en el desarrollo de software. Por Ejemplo:

<https://www.instagram.com/reels/DPNJZL2ESk0/>

Un ejemplo de aplicaciones inseguras por no utilizar seguridad en el software podría ser una tienda online que no utiliza validaciones, ya que, si la aplicación no realiza las validaciones de campos correspondientes, el usuario podría inyectar una instrucción SQL y ver la base de datos completa; desde usuarios, contraseñas o incluso borrarla al completo.

Por eso es muy importante utilizar las medidas de seguridad correspondientes.